

SIMATS ENGINEERING ASSESSMENT TOOL

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:20 Duration : 1 Hour

Scenario Based

Code of Conduct:

I Pusalapati Chandu(192372119)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

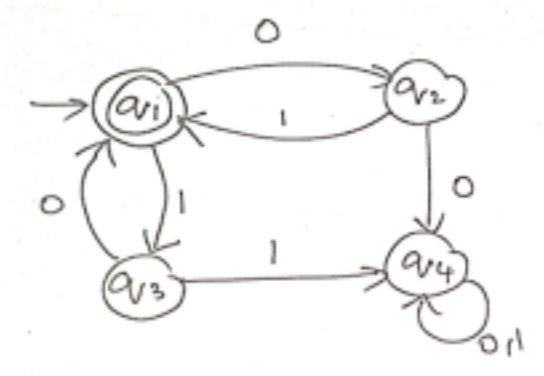
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

SIMATS ENGINEERING ASSESSMENT TOOL

1. Given a DFA used for validating a specific PIN pattern bbbc, bca, c, ca, caaaaa minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems

2. Demonstrate how a given finite automaton can be converted into a regular expression



3. Consider the following grammar

$S \rightarrow aSc | T | U | e$

$T \rightarrow bTc | e$

$U \rightarrow aUb | e$

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

4. I) Show that if L_1 and L_2 are regular then $(L_1 + L_2)$ also regular

ii) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify

Question 1

Given a DFA used for validating a specific PIN pattern **bbbc, bca, c, ca, caaaaaa** , minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems.

Step 1: Identify the Language Pattern

Observe the accepted PIN strings carefully: c, ca, bca, bbbc, and caaaaaa. In every one of these strings, a block of zero or more b's is followed by exactly one c, which is followed by zero or more a's. This is confirmed below:

Accepted String	b's (before c)	c	a's (after c)
c	0	1	0
ca	0	1	1
bca	1	1	1
bbbc	3	1	0
caaaaaa	0	1	6

Hence the underlying regular language is $L = b^*ca^*$ (any number of b's, followed by a single c, followed by any number of a's).

Step 2: Construct the (Non-Minimal) DFA

A DFA is first designed with one state used for every possible count of trailing a's, which introduces redundant states. Let q_0 be the start state, q_1 the state just after reading c, q_2 the state after reading one more a, q_3 after two or more a's, and q_d a dead/trap state for any invalid symbol.

Fig 1(a): Initial (Non-Minimal) DFA for $L = b^*ca^*$

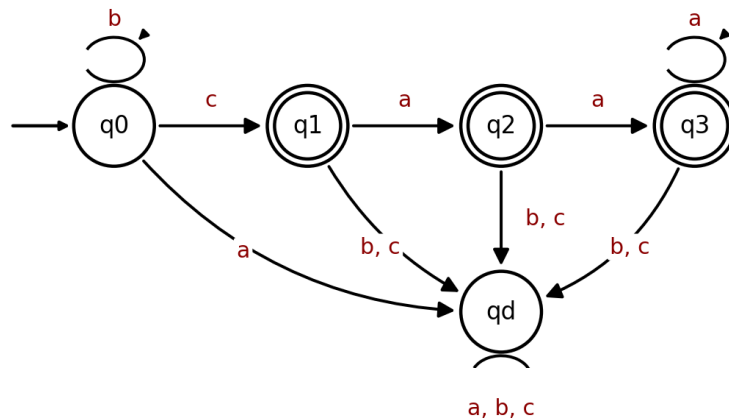


Figure 1(a): Initial DFA with redundant accepting states $q1, q2, q3$

State	on 'a'	on 'b'	on 'c'	Final?
$\rightarrow q0$	qd	q0	q1	No
q1	q2	qd	qd	Yes
q2	q3	qd	qd	Yes
q3	q3	qd	qd	Yes
qd	qd	qd	qd	No

Step 3: Minimize the DFA (Equivalence / Partition Method)

Partition 0: Split states into Final = $\{q1, q2, q3\}$ and Non-Final = $\{q0, qd\}$.

Check consistency of $\{q1, q2, q3\}$: on input 'b' and 'c', all three go to qd (same group). On input 'a': $q1 \rightarrow q2$, $q2 \rightarrow q3$, $q3 \rightarrow q3$ — all three stay within the SAME accepting group $\{q1, q2, q3\}$. So this group cannot be split further; $q1, q2$ and $q3$ are equivalent states.

Check consistency of $\{q0, qd\}$: on 'a', $q0 \rightarrow qd$ (leaves the group) while $qd \rightarrow qd$ (stays). Since their behaviour differs, this group must split into $\{q0\}$ and $\{qd\}$.

Final partition: $\{q0\}, \{q1, q2, q3\} \rightarrow$ merged as $Q1, \{qd\}$. This gives the minimized DFA with just 3 states.

Fig 1(b): Minimized DFA for $L = b^*ca^*$ ($q_1 = q_2 = q_3$ merged into Q_1)

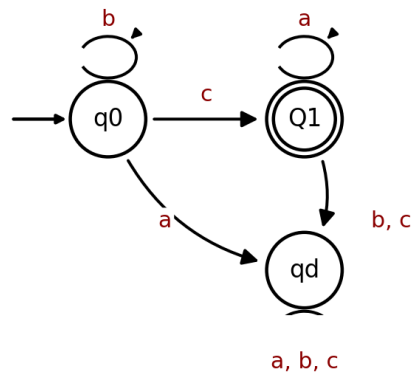


Figure 1(b): Minimized DFA — q_1, q_2, q_3 merged into a single state Q_1

State	on 'a'	on 'b'	on 'c'	Final?
$\rightarrow q_0$	q_d	q_0	Q_1	No
Q_1	Q_1	q_d	q_d	Yes
q_d	q_d	q_d	q_d	No

Fig 4: Flowchart of DFA Minimization
(Equivalence / Partition-Refinement Method)

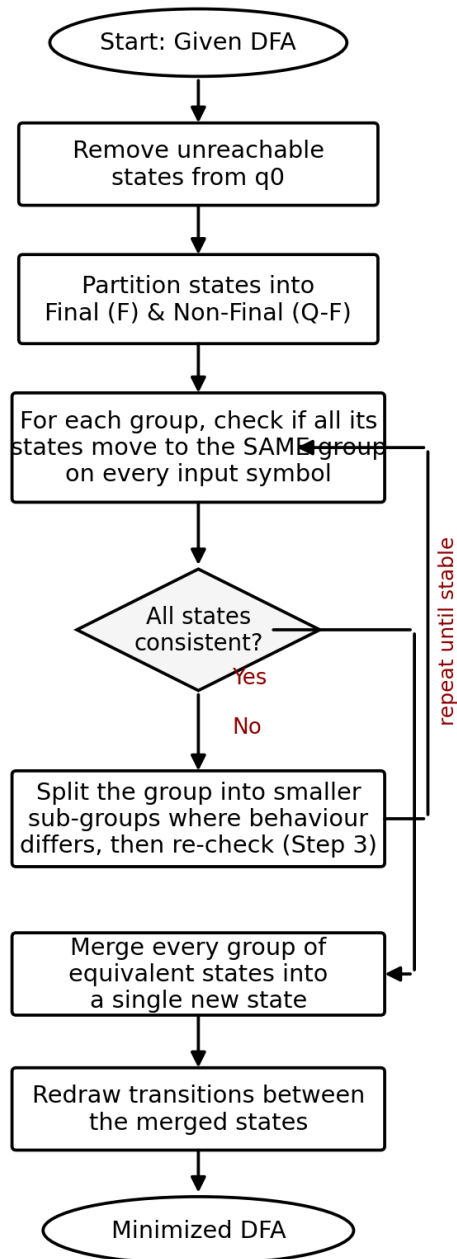


Figure 1(c): Flowchart of the DFA minimization procedure used above

Step 4: How State Reduction Improves Performance in Embedded Systems

- Smaller memory footprint: Fewer states mean a smaller state-transition table, which is critical in embedded controllers with limited ROM/RAM (e.g. a PIN-pad microcontroller).
- Faster execution: Each input symbol needs only one table look-up; fewer states mean the look-up table fits in cache/registers, reducing PIN validation latency.
- Lower power consumption: Reduced logic (fewer flip-flops in a hardware FSM implementation) directly lowers switching power — important for battery-operated embedded/IoT devices.
- Simpler hardware realization: When the DFA is synthesized as a digital circuit, fewer states reduce the number of flip-flops and gates required, lowering chip area and cost.
- Easier verification & certification: A minimal DFA is simpler to formally verify for correctness and security, which matters for PIN/authentication modules that must be certified.
- Reduced complexity for maintenance: Fewer equivalent states make future updates to the accepted PIN patterns easier to reason about and less error-prone.

Question 2 (5 Marks)

Demonstrate how a given finite automaton can be converted into a regular expression.

Step 1: Read the Given Automaton

The hand-drawn diagram is interpreted as a DFA with states A(=av1, start & final), B(=av2), C(=av3) and D(=av4, a dead/trap state). (Note: if any transition below differs from your original sketch, the same state-elimination method still applies — only the final expression would change.)

Fig 2: Finite Automaton (as interpreted from the given diagram)

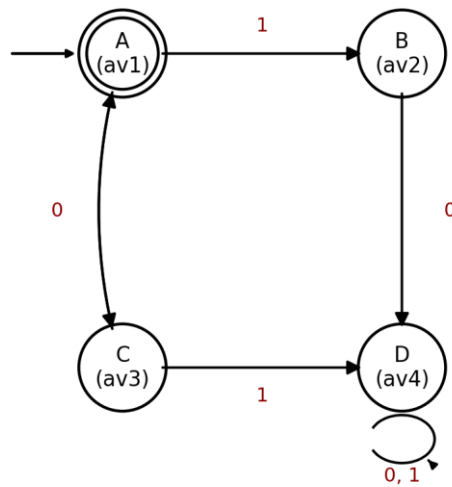


Figure 2: Automaton interpreted from the given diagram

State	on 0	on 1	Start/Final
A	C	B	Start & Final
B	D	A	—
C	A	D	—
D	D	D	Dead state

Step 2: General Method — State Elimination

Fig 5: Flowchart – Converting a Finite Automaton to a Regular Expression (State Elimination Method)

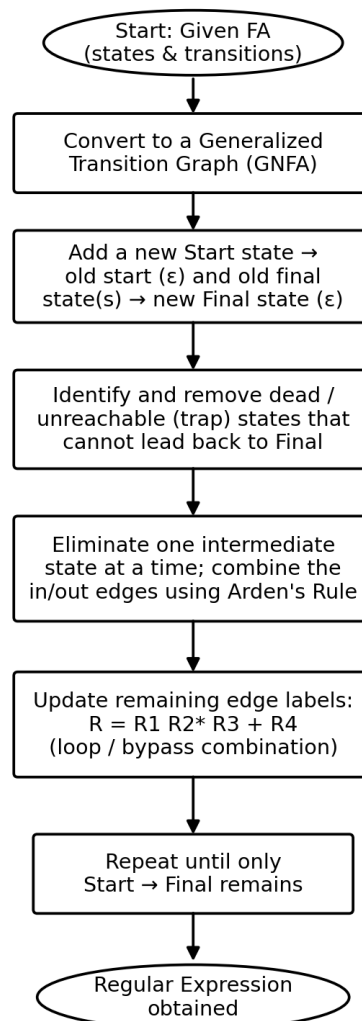


Figure 3: Flowchart of the State Elimination Method

Step 3: Apply the Method

Drop state D. D is a non-final trap state that never leads back to A, so no string that enters D can ever be accepted; it contributes nothing to the language and can be safely removed.

The remaining diagram now has only two ways to leave A and return to A:

- $A \rightarrow C$ on 0, then $C \rightarrow A$ on 0 $\Rightarrow$ the round trip reads the string '00'.

- $A \rightarrow B$ on 1, then $B \rightarrow A$ on 1 $\Rightarrow$ the round trip reads the string '11'.

Since A is both the start and the only final state, every accepted string is formed by repeating these two independent loops, in any order, any number of times (including zero times, which gives the empty string ϵ , also accepted because A is final).

Step 4: Final Regular Expression

$$R = (00 + 11)^*$$

This expression exactly captures all strings that can be decomposed into blocks of '00' and '11' repeated any number of times in any order — precisely the set of walks that leave A and return to A.

Question 3 (5 Marks)

Consider the grammar:

$$S \rightarrow aSc \mid T \mid U \mid \epsilon$$

$$T \rightarrow bTc \mid \epsilon$$

$$U \rightarrow aUb \mid \epsilon$$

A) What is the language defined by the grammar?

First analyse the helper non-terminals in isolation:

- $T \rightarrow bTc \mid \epsilon$ generates $L(T) = \{ b^m c^m : m \geq 0 \}$ (equal b's and c's).
- $U \rightarrow aUb \mid \epsilon$ generates $L(U) = \{ a^n b^n : n \geq 0 \}$ (equal a's and b's).

$S \rightarrow aSc \mid T \mid U \mid \epsilon$ wraps k matching a's and c's around T, U, or ϵ :

- Using T: $S \Rightarrow a^k T c^k = a^k (b^m c^m) c^k = a^k b^m c^{(m+k)}$ — here $(\#c) - (\#a) = \#b$.
- Using U: $S \Rightarrow a^k U c^k = a^k (a^n b^n) c^k = a^{(k+n)} b^n c^k$ — here $(\#a) - (\#c) = \#b$.
- Using ϵ directly: $S \Rightarrow a^k c^k$ — the special case $\#a = \#c$, $\#b = 0$.

Combining all three cases into one closed form:

$$L(S) = \{ a^i b^j c^l : i, j, l \geq 0 \text{ and } |i - l| = j \}$$

In words: the language consists of all strings of the form $a^*b^*c^*$ in which the number of b's exactly equals the (absolute) difference between the number of a's and the number of c's.

B) Derivation Tree for the word 'aaabbc'

For aaabbc: $i = 3$ a's, $j = 2$ b's, $l = 1$ c. Check: $|i - l| = |3 - 1| = 2 = j$ ✓, and $i > 1$ so this string is produced through the U-branch with $k = 1$ (one aSc wrap) and $n = 2$ (two aUb expansions).

Leftmost derivation:

$$S \Rightarrow aSc \Rightarrow aUc \Rightarrow a(aUb)c \Rightarrow aa(aUb)bc \Rightarrow aaa(\epsilon)bbc \Rightarrow aaabbc$$

Fig 3: Parse Tree for the string a a a b b c (via $S \Rightarrow aSc \Rightarrow aUc \Rightarrow \dots$)

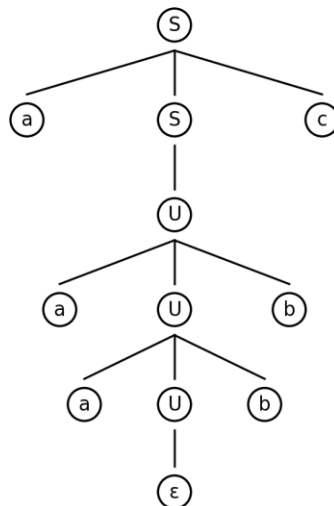


Figure 4: Parse tree for 'aaabbc' — reading the leaves left to right gives $a \cdot a \cdot a \cdot b \cdot b \cdot c$

C) Is the grammar ambiguous? Justify.

A grammar is ambiguous if some string in its language has two or more distinct parse trees (derivations). Consider the empty string ϵ . It can be derived in three structurally different ways from S:

- $S \Rightarrow \epsilon$ (directly, using $S \rightarrow \epsilon$)
- $S \Rightarrow T \Rightarrow \epsilon$ (using $S \rightarrow T$, then $T \rightarrow \epsilon$)
- $S \Rightarrow U \Rightarrow \epsilon$ (using $S \rightarrow U$, then $U \rightarrow \epsilon$)

These three derivations correspond to three distinct parse trees for the same string ϵ (they have different root-to-leaf structures, even though the yield is identical). Since at least one string has more than one parse tree, the grammar is AMBIGUOUS.

Question 4 (5 Marks)

I) Show that if L_1 and L_2 are regular, then $(L_1 \cup L_2)$ is also regular.

Proof (Closure under Union), using regular expressions: Since L_1 is regular, by definition there exists a regular expression r_1 such that $L(r_1) = L_1$. Similarly there exists r_2 with $L(r_2) = L_2$. By the recursive definition of regular expressions, if r_1 and r_2 are regular expressions, then $(r_1 + r_2)$ is also a regular expression, and its language is $L(r_1 + r_2) = L(r_1) \cup L(r_2) = L_1 \cup L_2$. Since $(L_1 \cup L_2)$ is the language of a regular expression, $L_1 \cup L_2$ is regular by definition. ■

Equivalent Constructive Proof (via NFA with ϵ -moves): Let $N_1 = (Q_1, \Sigma, \delta_1, p_0, F_1)$ accept L_1 and $N_2 = (Q_2, \Sigma, \delta_2, r_0, F_2)$ accept L_2 . Construct a new NFA N by adding a new start state q_0 with ϵ -transitions to p_0 and r_0 , keeping all final states of N_1 and N_2 as final states of N .

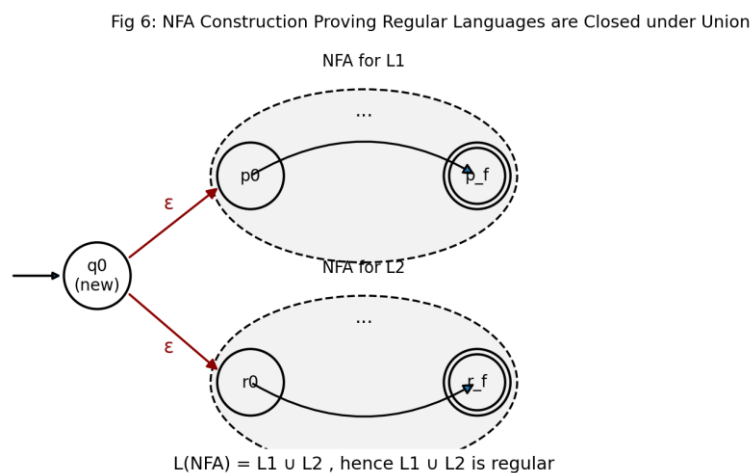


Figure 5: NFA construction proving closure of regular languages under union

Any string accepted by N_1 is accepted by N (via the ϵ -move into N_1), and likewise for N_2 ; N accepts nothing else. Hence $L(N) = L_1 \cup L_2$. Since a finite automaton exists for $L_1 \cup L_2$, it is regular. ■

II) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify.

Claim: No, this language is NOT regular.

Proof by the Pumping Lemma for regular languages:

- Suppose, for contradiction, that $L = \{a^p : p \text{ is prime}\}$ is regular. Then the Pumping Lemma guarantees a pumping length $n \geq 1$ such that every string $w \in L$ with $|w| \geq n$ can be split as $w = xyz$ with $|xy| \leq n$, $|y| \geq 1$, and $xy^iz \in L$ for every $i \geq 0$.

- Since there are infinitely many primes, choose a prime p with $p \geq n + 2$, and let $w = a^p$ (so $|w| = p \geq n$).
- By the lemma, $w = xyz$ with $|y| = k$, where $1 \leq k \leq n$, and $xy^iz \in L$ for all $i \geq 0$. Note $|xy^iz| = p + (i - 1)k$.
- Choose $i = p + 1$. Then $|xy^iz| = p + (p + 1 - 1)k = p + pk = p(1 + k)$.
- Since $p \geq 2$ and $k \geq 1$, we have $(1 + k) \geq 2$. Thus $p(1 + k)$ is a product of two integers each ≥ 2 , so it is composite — NOT prime.
- But the Pumping Lemma requires $xy^iz \in L$, i.e. its length must be prime. $p(1+k)$ is composite — a contradiction.

Since the assumption that L is regular leads to a contradiction, $L = \{ a^p : p \text{ is prime} \}$ is NOT regular. ■

Rubric for Calculation (Reference)

Criteria	Weightage	Level 4 (5)	Level 3 (4)	Level 2 (2–3)	Level 1 (0–1)
Conceptual Understanding	30	Clear & complete	Good, minor gaps	Basic, some misconceptions	Poor / incorrect
Accuracy of Answer	25	Completely correct	Mostly correct	Partially correct	Incorrect / irrelevant
Application of Knowledge	20	Correct application	Minor mistakes	Limited application	No / incorrect application
Completeness of Response	15	Covers all parts	Minor omissions	Covers some parts	Incomplete / missing
Clarity & Presentation	10	Clear & well-organized	Minor issues	Somewhat unclear	Difficult to understand